

slugify Slugify a string Useful for URLs, filenames, and IDs. It handles most major languages, including German (umlauts), Vietnamese, Arabic, Russian, and more. Install \$ npm install @sindresorhus/slugify 1	Usage import slugify from '@sindresorhus/slugify'; slugify('I ♥ Dogs'); //=> 'i-love-dogs' slugify(' Déjà Vu! '); //=> 'deja-vu' slugify('fooBar 123 \$#%'); //=> 'foo-bar-123' 2	slugify('я люблю единорогов'); //=> 'ya-lyublyu-edinorogov' API slugify(string, options?) string Type: string String to slugify. 3	options Type: object separator Type: string Default: '-' import slugify from '@sindresorhus/slugify'; 4
8 import slugify from '@sindresorhus/slugify'; slugify('fooBar', {decamelize: false}); //=> 'foo-bar' slugify('fooBar'); //=> 'foo-bar' 2	7 slugify('Déjà Vu!', {lowercase: false}); //=> 'Deja-Vu' decamelize Type: boolean Default: true Convert camelcase to separate words. Internally it does fooBar → foo bar. 3	9 import slugify from '@sindresorhus/slugify'; slugify('Déjà Vu!'); //=> 'deja-vu' lowercase Type: boolean Default: true Make the slug lowercase. 4	5 slugify('BAR and baz', {separator: '-'}); //=> 'bar-and-baz' slugify('BAR and baz', {separator: ''}); //=> 'barandbaz' 5
customReplacements Type: Array<string[]> Default: [['&', ' and '], ['🦄', ' unicorn '], ['♥', ' love ']] Add your own custom replacements. The replacements are run on the original string before any other transformations. This only overrides a default replacement if you set an item with the same key, like &. 6	10 import slugify from '@sindresorhus/slugify'; slugify('Foo@unicorn', {customReplacements: [['@', ' at ']]}); //=> 'fooatunicorn' Add a leading and trailing space to the replacement to have it separated by dashes: 7	11 import slugify from '@sindresorhus/slugify'; slugify('foo@unicorn', {customReplacements: [['@', ' at ']]}); //=> 'foo-at-unicorn' Another example: 8	12 import slugify from '@sindresorhus/slugify'; slugify('I love 🐶', {customReplacements: [['🐶', ' dogs']]}); //=> 'i-love-dogs' 9
16 slugify('foo-bar-', {preserveTrailingDash: true}); //=> 'foo-bar-' preserveCharacters Type: string[] Default: [] Preserve certain characters. It cannot contain the separator. For example, if you want to slugify URLs, but preserve the HTML fragment # character. 10	15 import slugify from '@sindresorhus/slugify'; slugify('foo-bar-'); //=> 'foo-bar' preserveTrailingDash Type: boolean Default: false If your string ends with a dash, it will be preserved in the slugified string. For example, using slugify on an input field preventing the user from writing a slug. 11	14 slugify('foo-bar', {preserveLeadingUnderscore: true}); //=> 'foo-bar' preserveLeadingUnderscore Type: boolean Default: false If your string starts with an underscore, it will be preserved in the slugified string. Sometimes leading underscores are intentional, for example, filenames representing hidden paths on a website. 12	13 import slugify from '@sindresorhus/slugify'; slugify(''); 13

<pre>import slugify from '@sindresorhus/ slugify'; slugify('foo_bar#baz', {preserveCharacters: ['#']}); //=> 'foo-bar#baz'</pre> slugifyWithCounter() Returns a new instance of slugify(string, options?) with a counter 17	to handle multiple occurrences of the same string. Example <pre>import {slugifyWithCounter} from '@sindresorhus/slugify'; const slugify = slugifyWithCounter(); slugify('foo bar'); //=> 'foo-bar'</pre> 18	<pre>slugify('foo bar'); //=> 'foo-bar-2' slugify.reset(); slugify('foo bar'); //=> 'foo-bar'</pre> 19	Use-case example of counter If, for example, you have a document with multiple sections where each subsection has an example. <pre>## Section 1 ### Example ## Section 2 ### Example</pre> 20
24	23 slugify.reset(); slugify('foo bar'); //=> 'foo-bar' Related • slugify-cli - CLI for this module • transliterate - Convert Unicode characters to Latin characters using transliteration • filenameify - Convert a string to a valid safe filename 25	Example <pre>import {slugifyWithCounter} from '@sindresorhus/slugify'; const slugify = slugifyWithCounter(); slugify('foo bar'); //=> 'foo-bar' slugify('foo bar'); //=> 'foo-bar-2'</pre> 22	You can then use slugifyWithCounter() to generate unique HTML id's to ensure anchors will link to the right headline. slugify.reset() Reset the counter 21
25	26	27	28
32	31	30	29